



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ

KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Projekt sztucznego horyzontu z AHRS

Autor:

Tomasz Kurowski

Kierunek studiów:

Informatyka Techniczna

Kraków, 2026

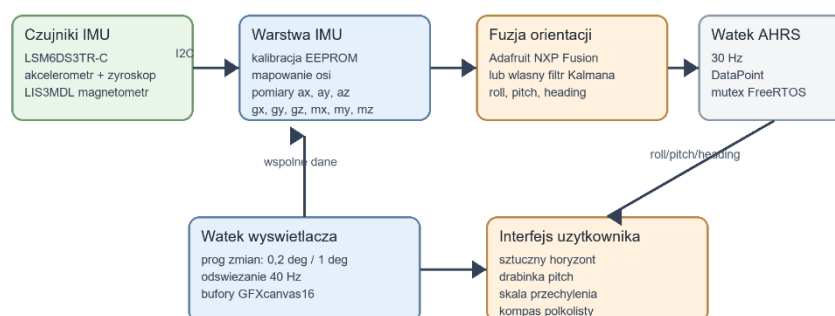
Spis treści

1. Wprowadzenie i cele.....	2
2. Proces tworzenia projektu.....	3
3. Wykorzystane komponenty i urządzenia.....	4
4. Konstrukcja mechaniczna, schemat elektryczny i zasilanie	9
5. Oprogramowanie, wielowątkowość i architektura projektu.....	11
6. Kalibracja sensorów oraz transformacja osi	13
7. Akwizycja danych i fuzja przy użyciu Adafruit AHRS	16
8. Autorski Filtr Kalmana	18
9. Interfejs użytkownika na wyświetlaczu	20
10. Podsumowanie i dalsze kierunki rozwoju.....	22
Załączniki.....	23
Wykaz rysunków.....	24
Wykaz tabel.....	25
Wykaz fragmentów kodu	26
Wykaz równań	27

1. Wprowadzenie i cele

Przedstawiony projekt systemu AHRS (ang. Attitude and Heading Reference System) jest przenośnym układem do wyznaczania orientacji przestrzennej oraz prezentowania jej w postaci sztucznego horyzontu i kompasu. Urządzenie zbiera dane z 3 osiowych akcelerometru, żyroskopu i magnetometru, wykonuje fuzję sensoryczną, a wynikowe kąty pochylenia, przechylenia oraz kurs wyświetla na ekranie LCD.

Architektura przepływu danych w projekcie AHRS



Rys. 1. Uproszczona architektura przepływu danych w projekcie

Oprogramowanie projektu wykorzystuje bibliotekę Adafruit AHRS, a dokładniej klasę `Adafruit_NXP SensorFusion` przeprowadzającą fuzję sensorów za pomocą filtru Kalmana. Repozytorium zawiera również autorską bibliotekę filtru Kalmana, która może zastąpić gotowy filtr.

Główne cele projektu:

- odczyt orientacji przestrzennej z płytki IMU 9-DOF;
- prezentacja przechylenia, pochylenia i kursu w formie czytelnej dla użytkownika;
- oddzielenie wątku fuzji danych od wątku wyświetlania za pomocą mechanizmów FreeRTOS;
- zachowanie możliwości przejścia z gotowej biblioteki Adafruit AHRS na autorski filtr Kalmana;
- opracowanie obudowy oraz schematów elektrycznych umożliwiających wykonanie fizycznego prototypu.

2. Proces tworzenia projektu

Projekt był rozwijany etapowo, od warstwy mechanicznej do oprogramowania i kalibracji. Takie podejście było korzystne, ponieważ najpierw określono gabaryty obudowy, położenie ekranu, płytki Arduino, modułu IMU, przełącznika oraz koszyka baterii. Dopiero po ustaleniu geometrii można było bezpiecznie przejść do połączeń elektrycznych i kodu.

1. Zaprojektowano modele 3D obudowy w SolidWorks: korpus, front oraz pokrywę baterii.
2. Wydrukowano elementy na drukarce BambuLab X1 Carbon i zweryfikowano spasowanie płytki, wyświetlacza oraz elementów zasilania.
3. Wykonano część elektryczną, lutując przewody i łącząc moduły wtyczkami zgodnie ze schematami ArduinoBoard, PowerSupply oraz Prototyping_Area.
4. Napisano oprogramowanie mikrokontrolera: inicjalizację czujników, fuzję danych, zadania FreeRTOS i rysowanie interfejsu.
5. Przeprowadzono kalibrację sensorów narzędziami Adafruit Sensor Lab i MotionCal, a uzyskane parametry zapisano w pamięci EEPROM Arduino Nano ESP32.

W praktyce etapy te były od siebie zależne: orientacja IMU wynikała z rozmieszczenia modułu w obudowie, metoda zasilania z użycia baterii AA (co miało na celu umożliwienie łatwego przenoszenia modułu), a podział na watki z potrzeby jednoczesnego odświeżania grafiki i liczenia orientacji.

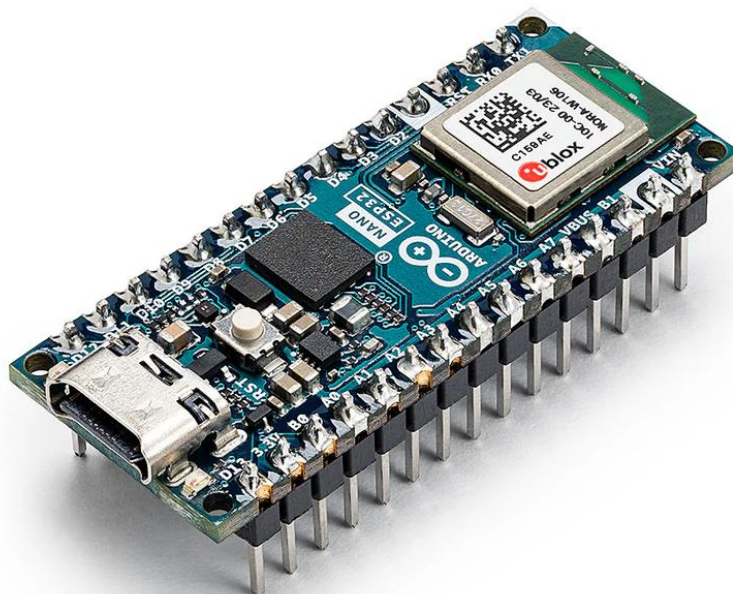
3. Wykorzystane komponenty i urządzenia

Zestaw elementów przewidzianych do budowy urządzenia został dobrany tak, aby połączyć niewielką płytkę sterującą, czujnik inercyjny i kolorowy wyświetlacz z niezależnym układem zasilania.

Tab. 1. Tabela komponentów projektu

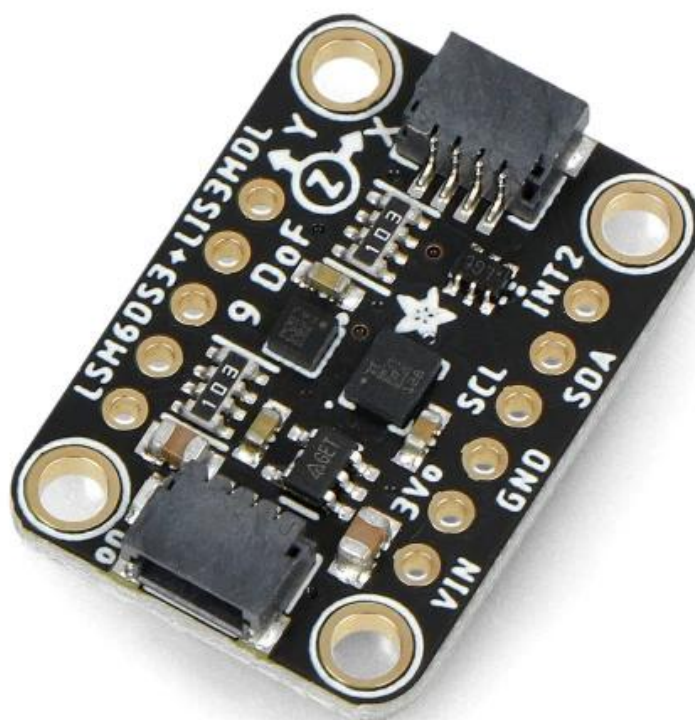
Komponent	Rola w projekcie
Arduino Nano ESP32	Główny mikrokontroler uruchamiający szkic AHRS.ino.
IMU 9-DOF	Zestaw LSM6DS3TR-C oraz LIS3MDL: akcelerometr, żyroskop i magnetometr.
Wyświetlacz ST7789 240 x 320	Ekran do prezentacji sztucznego horyzontu i kompasu.
Adapter dla Arduino Nano	Ułatwia wyprowadzenie pinów Arduino.
Przetwornica step-up	Element układu zasilania, umożliwiający użycie baterii AA.
Przełącznik suwakowy	Mechaniczny element włączania lub odcinania zasilania urządzenia.
Obudowa drukowana 3D	Modele STL i SolidWorks w katalogu Case/3dPrint oraz Case/3dModels.

Arduino Nano ESP32 jest główna jednostka obliczeniowa projektu (Rys. 2). Uruchamia szkic AHRS.ino, obsługuje I2C dla IMU, SPI dla wyświetlacza ST7789 oraz dwa zadania FreeRTOS (renderowanie grafiki i fuzja sensorów) uruchomione na oddzielnych rdzeniach.



Rys. 2. Płytki Arduino Nano ESP32 wykorzystana jako jednostka sterująca

Moduł Adafruit LSM6DS3TR-C + LIS3MDL 9DoF IMU (Rys. 3) łączy dwa układy: LSM6DS3TR-C, czyli akcelerometr i żyroskop, oraz LIS3MDL, czyli magnetometr. Akcelerometr dostarcza informacji o kierunku wektora grawitacji, żyroskop mierzy prędkości kątowe, a magnetometr umożliwia wyznaczenie kursu względem pola magnetycznego ziemi. Dopiero fuzja tych pomiarów daje stabilną orientację: żyroskop zapewnia szybka reakcję, akcelerometr ogranicza dryf przechylenia i pochylenia, a magnetometr koryguje kurs.



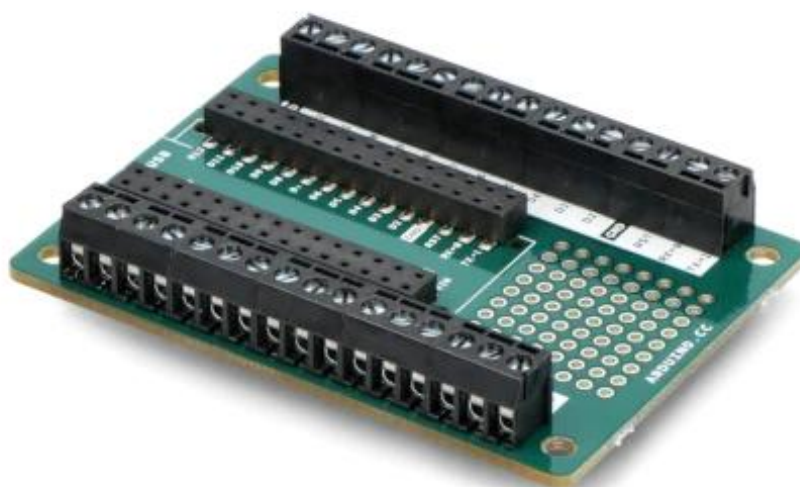
Rys. 3. Moduł IMU dostarczający pomiary inercyjne i magnetyczne

Wyświetlacz LCD TFT ST7789 o rozdzielczości 240 x 320 px prezentuje na swojej górnej części sztuczny horyzont, a na dolnej półkolisty kompas. Grafika jest przygotowywana w buforach GFXcanvas16, a następnie wyświetlana na ekranie.



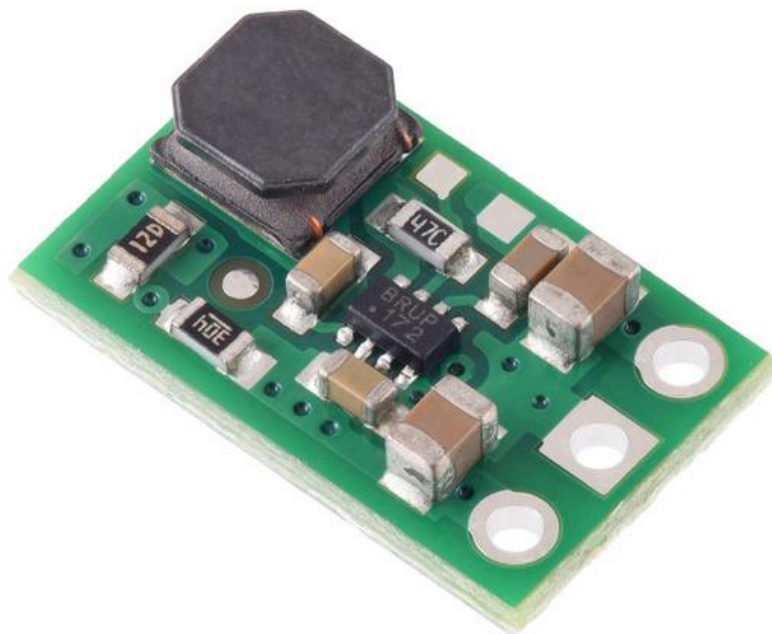
Rys. 4. Wyświetlacz LCD TFT używany do prezentacji wskazań AHRS

Adapter ułatwia montaż płytki oraz rozłożenie przewodów i komponentów elektrycznych takich jak wtyczki i oporniki dzięki obszarowi prototypowemu. Dodatkowo wyprowadza piny Arduino na złącza śrubowe.



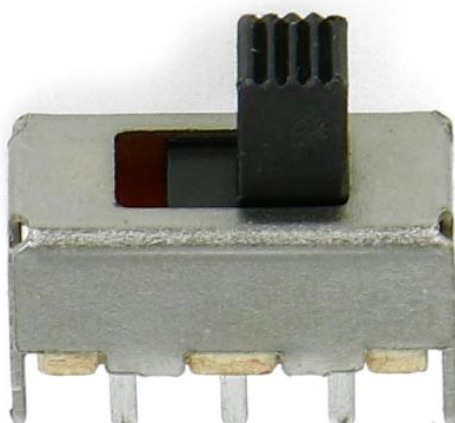
Rys. 5. Adapter dla Arduino Nano ESP32

Przetwornica Pololu U3V16F3 podnosi napięcie z zapewnione przez dwie baterie AA umieszczone w koszyku na górze obudowy z 3 do 3,3 V wymaganych przez Arduino Nano ESP32 i peryferia.



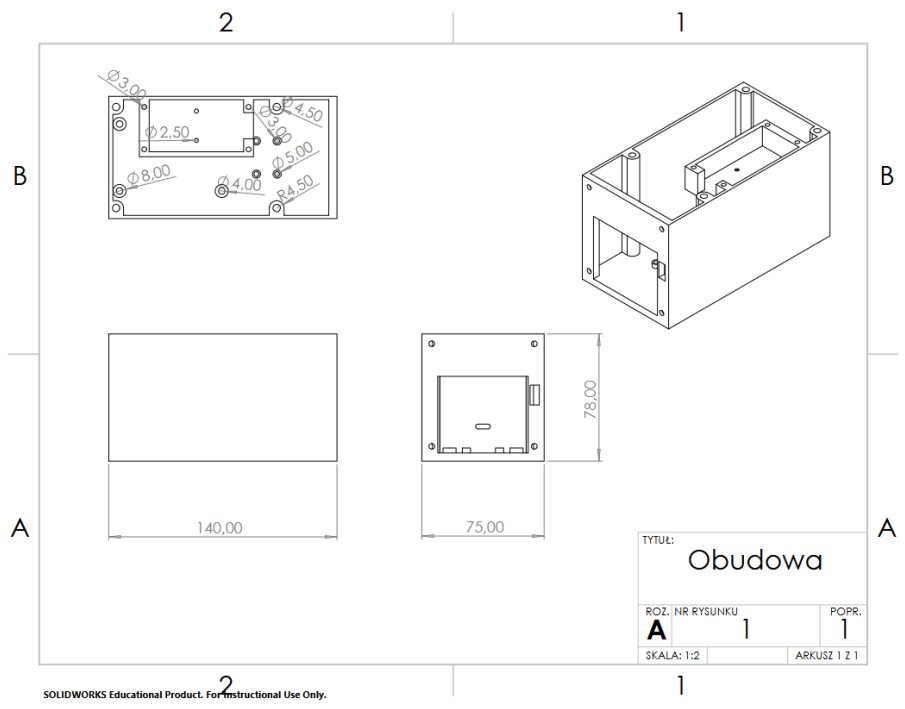
Rys. 6. Przetwornica step-up Pololu U3V16F3

Przełącznik suwakowy pełni rolę włącznika urządzenia. Po przełączeniu doprowadza zasilanie z baterii do przetwornicy step-up.



Rys. 7. Przełącznik suwakowy włączający sztuczny horyzont

Obudowa urządzenia została zaprojektowana w programie SolidWorks a następnie wydrukowana z filamentu PLA na drukarce 3D BambuLab X1 Carbon.

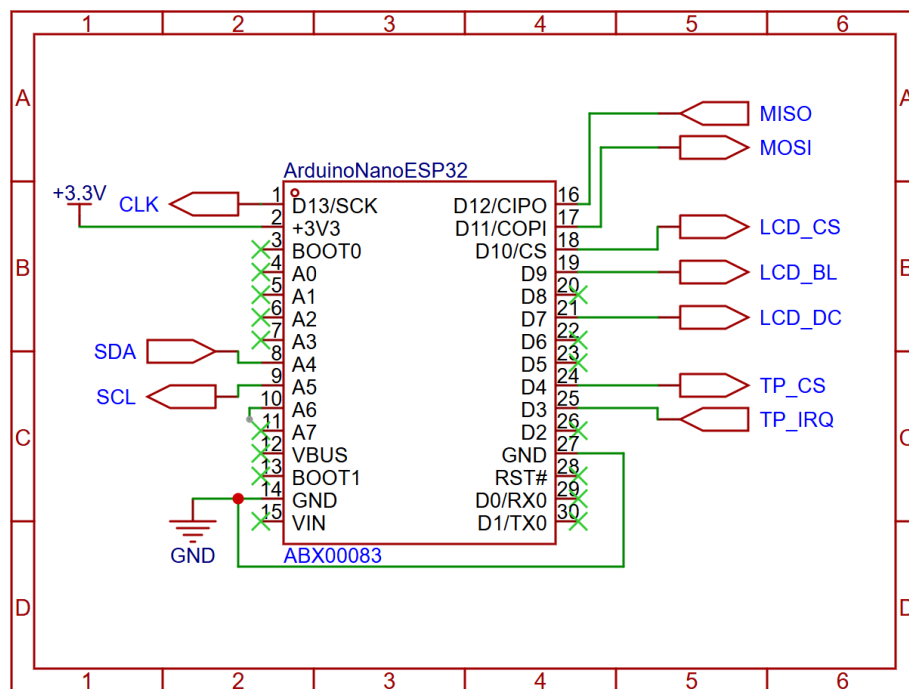


Rys. 8. Rysunek wewnętrznej części obudowy

4. Konstrukcja mechaniczna, schemat elektryczny i zasilanie

Warstwa sprzętowa projektu została rozdzielona na schemat płytki Arduino, zasilania oraz obszar prototypowy. Taki podział ułatwia weryfikację połączeń: osobno można analizować linie mikrokontrolera, przetwornice i elementy wykonawcze montowane na płytce prototypowej.

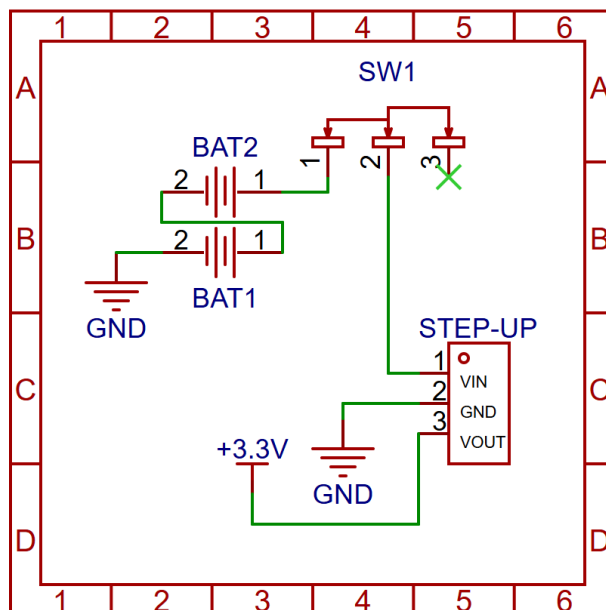
Z płytki Arduino wyprowadzono piny odpowiedzialne za komunikację przy użyciu protokołu I²C (używany przez moduł IMU) oraz SPI (komunikacja z wyświetlaczem).



Rys. 9. Schemat połączeń płytki Arduino Nano ESP32

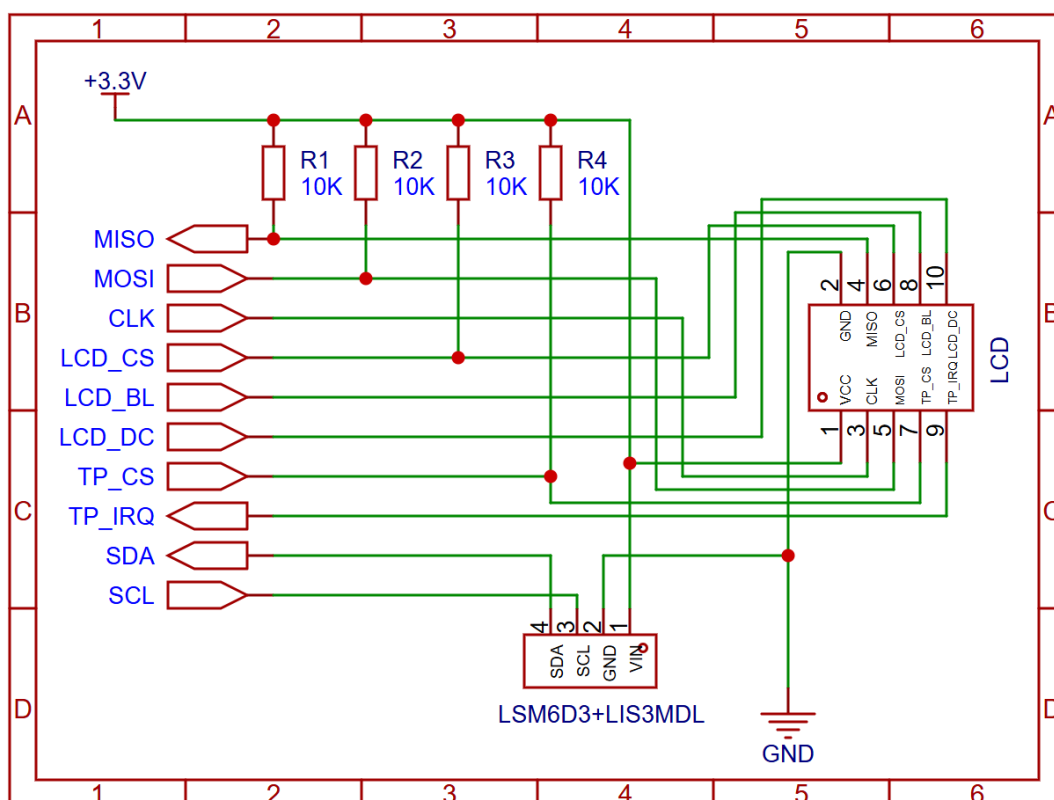
Sztuczny horyzont jest zasilany z dwóch baterii AA 1,5 V połączonych szeregowo. Nominalne napięcie takiego pakietu wynosi 3 V, dlatego nie wystarcza do bezpośredniego zasilania Arduino Nano ESP32 pracującego z logiką 3,3 V. W torze zasilania zastosowano przetwornice step-up Pololu U3V16F3, która stabilizuje napięcie wyjściowe na poziomie 3,3 V.

Przełącznik suwakowy odcina lub doprowadza napięcie z koszyka baterii do przetwornicy. Wyjście 3,3 V przetwornicy zasila płytkę przez pin 3.3V. Aby umożliwić takie zasilanie i nie łączyć zewnętrznej przetwornicy z wewnętrznym regulatorem płytki, przerwano zworkę na spodzie Arduino Nano ESP32 oznaczoną jako "3.3V".



Rys. 10. Schemat zasilania projektu

Obszar prototypowy pozwolił na rozmieszczenie wtyczek pozwalających połączyć moduły a także oporników pełniących rolę Pull-Up dla pinów wykorzystywanych do komunikacji przez SPI.



Rys. 11. Schemat obszaru prototypowego

5. Oprogramowanie, wielowątkowość i architektura projektu

Tab. 2. Tabela elementów oprogramowania

Element oprogramowania	Rola w projekcie
taskAHRS	Odczyt IMU, fuzja danych, zapis DataPoint do wspólnej struktury przez attitudeMutex.
taskDisplay	Porównanie zmian kątów, aktualizacja sztucznego horyzontu i kompasu.
IMU	Inicjalizacja LSM6DS3TR-C i LIS3MDL, kalibracja, transformacja osi, wywołanie filtru (gdy używany filtr z biblioteki Adafruit AHRS).
Display	Rysowanie horyzontu i kompasu.
KalmanFilter	Predykcja i aktualizacja stanu i fuzja danych. Własna klasa, używana w zamian za Adafruit AHRS
linAlg	Minimalna algebra liniowa: mnożenie macierzy, transpozycja, wyznacznik i macierz odwrotna.

Najważniejsze stałe konfiguracyjne:

- FILTER_UPDATE_RATE_HZ = 30 - częstotliwość pracy filtru orientacji,
- ATT_REFRESH_RATE_HZ = 40 - odświeżanie warstwy wyświetlania,
- ATT_THRESHOLD_DEG = 0.2 - minimalna zmiana przechylenia/pochylenia wymuszająca przerysowanie horyzontu,
- HDG_THRESHOLD_DEG = 1.0 - minimalna zmiana kursu wymuszająca przerysowanie kompasu,
- PIXELS_PER_DEGREE = 7.0 - przelicznik kąta pochylenia na przesunięcie horyzontu.

Szkic Arduino tworzy obiekty Display, IMU, DeviceControl oraz KalmanFilter (gdy używany jest własny algorytm), a następnie uruchamia dwa zadania FreeRTOS na oddzielnych rdzeniach procesora mikrokontrolera (Fragment kodu 1).

```
SemaphoreHandle_t attitudeMutex;

void taskAHRS(void *pvParameters);
void taskDisplay(void *pvParameters);

void setup()
{
    pinMode(LCD_CS, OUTPUT);
    digitalWrite(LCD_CS, HIGH);
    pinMode(TP_CS, OUTPUT);
    digitalWrite(TP_CS, HIGH);
    pinMode(LCD_BCKLIT, OUTPUT);
    digitalWrite(LCD_BCKLIT, HIGH);
}
```

```
deviceControl.powerOn(kalmanData);

attitudeMutex = xSemaphoreCreateMutex();

xTaskCreatePinnedToCore(taskAHRS, "AHRS", 8192, NULL, 2, NULL, 1);
xTaskCreatePinnedToCore(taskDisplay, "DISPLAY", 16384, NULL, 1, NULL, 0);
}
```

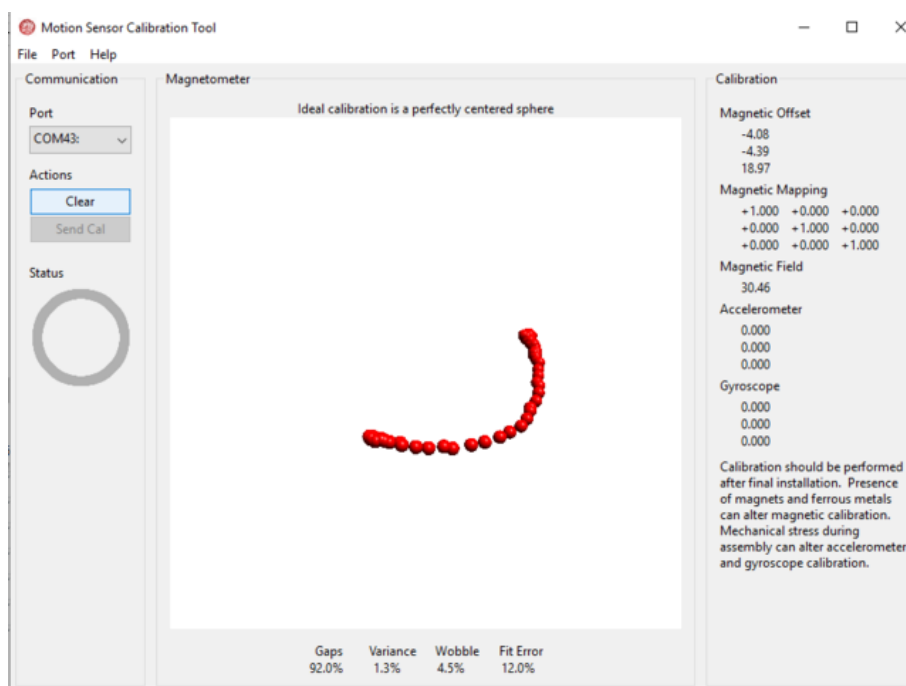
Fragment kodu 1. Tworzenie zadań na oddzielnych rdzeniach

Zadanie AHRS odpowiada za odczyt czujników i wyznaczenie orientacji. Zadanie DISPLAY pracuje niezależnie, pobiera ostatnie dane z użyciem mutexu i odświeża tylko te elementy ekranu, które zmieniły się ponad próg, aby uniknąć ponownego renderowania gdy nie jest to konieczne.

Najważniejszym założeniem programowym jest oddzielenie części pomiarowo-filtrującej od części graficznej. Zadanie taskAHRS jest przypięte do rdzenia 1 i ma wyższy priorytet, ponieważ odpowiada za cykliczny odczyt IMU oraz fuzję danych. Zadanie taskDisplay jest przypięte do rdzenia 0 i może wykonywać kosztowniejsze rysowanie bez blokowania próbkowania.

6. Kalibracja sensorów oraz transformacja osi

Przed docelową pracą układu wykonano kalibrację czujników za pomocą narzędzi Adafruit Sensor Lab oraz MotionCal. Kalibracja jest konieczna, ponieważ surowe pomiary zawierają przesunięcia zera żyroskopu, zakłócenia magnetyczne stałe (hard-iron) oraz zniekształcenia skali i osi magnetometru (soft-iron).



Rys. 12. Interfejs programu MotionCal

Uzyskano poniższe wyniki kalibracji magnetometru:

Calibration			
Magnetic Offset			
57.32			
-25.83			
-62.12			
Magnetic Mapping			
+0.948	-0.009	-0.007	
-0.009	+0.999	+0.003	
-0.007	+0.003	+1.056	
Magnetic Field			
55.53			
Accelerometer			
0.000			
0.000			
0.000			
Gyroscope			
0.000			
0.000			
0.000			
Calibration should be performed after final installation. Presence of magnets and ferrous metals can alter magnetic calibration. Mechanical stress during assembly can alter accelerometer and gyroscope calibration.			

Rys. 13. Parametry kalibracji magnetometru
Źródło: opracowanie własne

Kalibracja żyroskopu została wykonana narzędziem Adafruit Sensor Lab. W pliku Gyro_calibration.txt zapisano trzy serie pomiarów przesunięcia zera w rad/s, a do EEPROM wpisano średnie wartości:

```
Adafruit Sensor Lab Gyro Calibration Results
-----

Final zero rate offset in radians/s:
-0.0012, -0.0049, -0.0367

Final zero rate offset in radians/s:
-0.0012, -0.0061, -0.0367

Final zero rate offset in radians/s:
-0.0012, -0.0055, -0.0373

AVERAGES:
-----
-0.0012, -0.0055, -0.0369
```

Rys. 14. Dane kalibracji żyroskopów

Dane kalibracji magnetometru i żyroskopu zostały następnie wpisane do szkicu sensor_calibration_write.ino, który korzysta z Adafruit_Sensor_Calibration_EEPROM i zapisuje parametry w pamięci płytki.

```
// in uTesla
cal.mag_hardiron[0] = 57.32;
cal.mag_hardiron[1] = -25.83;
cal.mag_hardiron[2] = -62.12;

// in uTesla
cal.mag_softiron[0] = 0.948;
cal.mag_softiron[1] = -0.009;
cal.mag_softiron[2] = -0.007;
cal.mag_softiron[3] = -0.009;
cal.mag_softiron[4] = 0.999;
cal.mag_softiron[5] = 0.003;
cal.mag_softiron[6] = -0.007;
cal.mag_softiron[7] = 0.003;
cal.mag_softiron[8] = 1.056;

// Earth total magnetic field strength in uTesla (dependent on location and time of the
year),
cal.mag_field = 55.53;

// in Radians/s
cal.gyro_zerorate[0] = -0.0012;
cal.gyro_zerorate[1] = -0.0055;
cal.gyro_zerorate[2] = -0.0369;
```

Fragment kodu 2. Dane kalibracji w szkicu służącym do jej zapisu w pamięci EEPROM

W docelowym programie kalibracja jest odczytywana przy starcie a następnie stosowana osobno do pomiarów magnetometru, akcelerometru i żyroskopu.

Modul IMU nie jest zamontowany w sposób zgodny z osiami urządzenia/statku powietrznego. Z tego powodu konieczna jest zamiana osi oraz znaków. Jest to etap równie ważny jak sama kalibracja: bez niego przechył, pochylenie lub kurs byłyby interpretowane w niewłaściwych kierunkach.

```
kf::Measurement measurement = kf::Measurement{
    .millis = millis(),
    .ax = -accel.acceleration.y,
    .ay = accel.acceleration.x,
    .az = accel.acceleration.z,
    .gx = -gyro.gyro.y * SENSORS_RADS_TO_DPS,
    .gy = gyro.gyro.x * SENSORS_RADS_TO_DPS,
    .gz = gyro.gyro.z * SENSORS_RADS_TO_DPS,
    .mx = -mag.magnetic.y,
    .my = mag.magnetic.x,
    .mz = mag.magnetic.z,
    .temp = temp.temperature};
```

Fragment kodu 3. Uzgodnienie osi IMU

7. Akwizycja danych i fuzja przy użyciu Adafruit AHRS

Klasa IMU inicjalizuje dwa układy Adafruit: LSM6DS3TR-C oraz LIS3MDL. Po starcie ładowana jest kalibracja z EEPROM, ustawiane są zakresy pomiarowe i częstotliwości próbkowania. Akcelerometr i żyroskop pracują z częstotliwością 104 Hz, magnetometr w trybie ciągłym z szybkim odczytem, a magistrala I²C zostaje ustawiona na 400 kHz.

```
// set lowest range
lsm6ds.setAccelRange(LSM6DS_ACCEL_RANGE_2_G);
lsm6ds.setGyroRange(LSM6DS_GYRO_RANGE_250_DPS);
lis3mdl.setRange(LIS3MDL_RANGE_4_GAUSS);

// set slightly above refresh rate
lsm6ds.setAccelDataRate(LSM6DS_RATE_104_HZ);
lsm6ds.setGyroDataRate(LSM6DS_RATE_104_HZ);
lis3mdl.setDataRate(LIS3MDL_DATARATE_1000_HZ);
lis3mdl.setPerformanceMode(LIS3MDL_MEDIUMMODE);
lis3mdl.setOperationMode(LIS3MDL_CONTINUOUSMODE);

filter.begin(FILTER_UPDATE_RATE_HZ);
// timestamp = millis();

Wire.setClock(400000);
```

Fragment kodu 4. Ustawienia IMU

W funkcji IMU::read(...) dane są kalibrowane, a następnie osie są przemapowane do układu przyjętego w projekcie. Aktywna ścieżka wywołuje filter.update(...) z biblioteki Adafruit i pobiera przechylenie, pochylenie oraz kurs, który jest dodatkowo przesunięty o 90 stopni i zawinięty operacją fmod aby uzgodnić wskazania magnetometru rzeczywistym położeniem względem północy magnetycznej.

```
filter.update(measurement.gx, measurement.gy, measurement.gz,
              measurement.ax, measurement.ay, measurement.az,
              measurement.mx, measurement.my, measurement.mz);

dataPoint = kf::DataPoint{
    .millis = measurement.millis,
    .roll = filter.getRoll(),
    .pitch = filter.getPitch(),
    .heading = fmod(-filter.getYaw() - 90.0f, 360.0f)};
```

Fragment kodu 5. Aktualizacja filtru Kalmana i pobranie danych

Biblioteka Adafruit AHRS realizuje fuzję danych inercyjnych i magnetycznych w sposób typowy dla układów AHRS: szybkie, pomiary żyroskopu są korygowane wolniejszymi wskazaniami

akcelerometru i magnetometru. Adafruit_NXP SensorFusion bazuje na implementacji algorytmu filtru Kalmana, który utrzymuje orientację wewnętrznie w reprezentacji kwaternionowej. W przeciwieństwie do samych kątów Eulera, kwaternion nie ma osobliwości typu gimbal lock i dobrze nadaje się do całkowania prędkości kątowej z żyroskopu.

Korekcje z akcelerometru i magnetometru zmniejszają błąd narastający przez dryf żyroskopu: akcelerometr wskazuje kierunek grawitacji, a magnetometr dostarcza odniesienia dla kursu.

Biblioteka zwraca kąty Eulera wyliczone z wewnętrznego stanu kwaternionowego.

8. Autorski Filtr Kalmana

Autorska implementacja filtra znajduje się w katalogu /kalmanFilter. Obecnie jest ona przygotowana jako alternatywa dla biblioteki Adafruit. Obiekt KalmanFilter jest tworzony w AHRS.ino, ale wywołanie kalmanRun(...) pozostaje zakomentowane.

Stan filtra jest sześciowymiarowy. Pierwsze trzy składowe reprezentują kąty Eulera, a kolejne trzy składowe reprezentują estymowane składowe dryfu żyroskopu:

$$x(t) = \begin{bmatrix} roll \\ pitch \\ yaw \\ b_{gx} \\ b_{gy} \\ b_{gz} \end{bmatrix} \quad (1)$$

Wektor sterowania zawiera pomiary żyroskopu:

$$u(t) = \begin{bmatrix} g_x \\ g_y \\ g_z \end{bmatrix}. \quad (2)$$

Wektor pomiarów zawiera dane z akcelerometrów oraz magnetometru:

$$z(t) = \begin{bmatrix} roll_{acc} \\ pitch_{acc} \\ yaw_{mag} \end{bmatrix}. \quad (3)$$

Macierz przejścia:

$$F = \begin{bmatrix} 1 & 0 & 0 & -\Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & -\Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & -\Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (4)$$

Macierz sterowania:

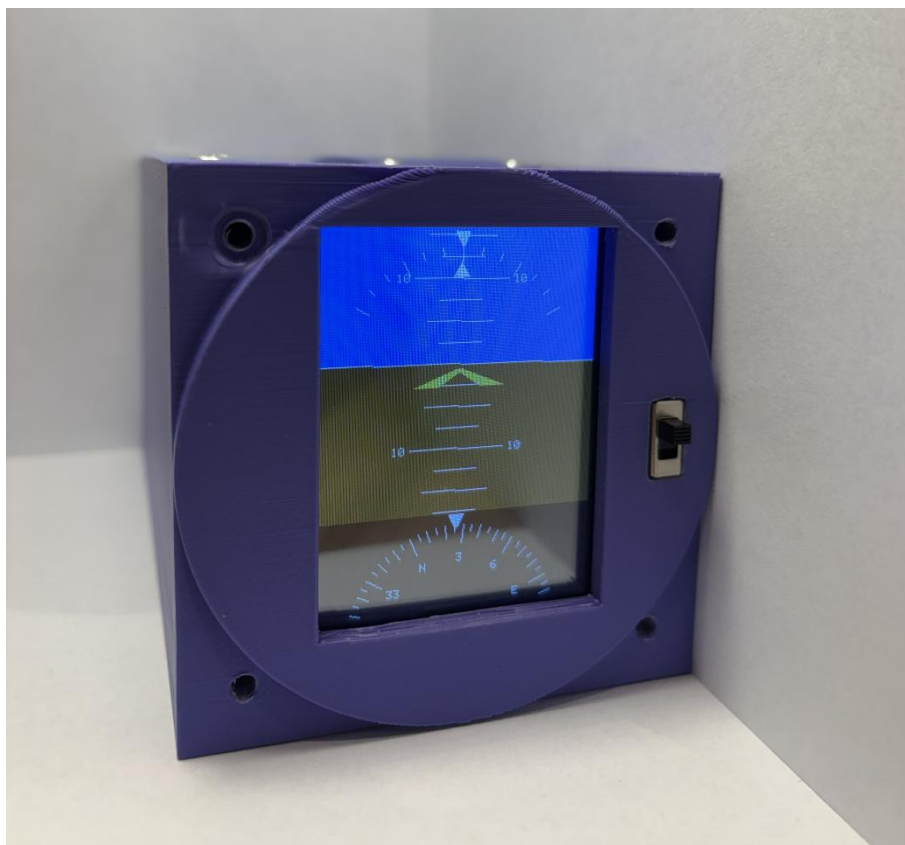
$$G = \begin{bmatrix} \Delta t & 0 & 0 \\ 0 & \Delta t & 0 \\ 0 & 0 & \Delta t \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}. \quad (5)$$

Macierz obserwacji:

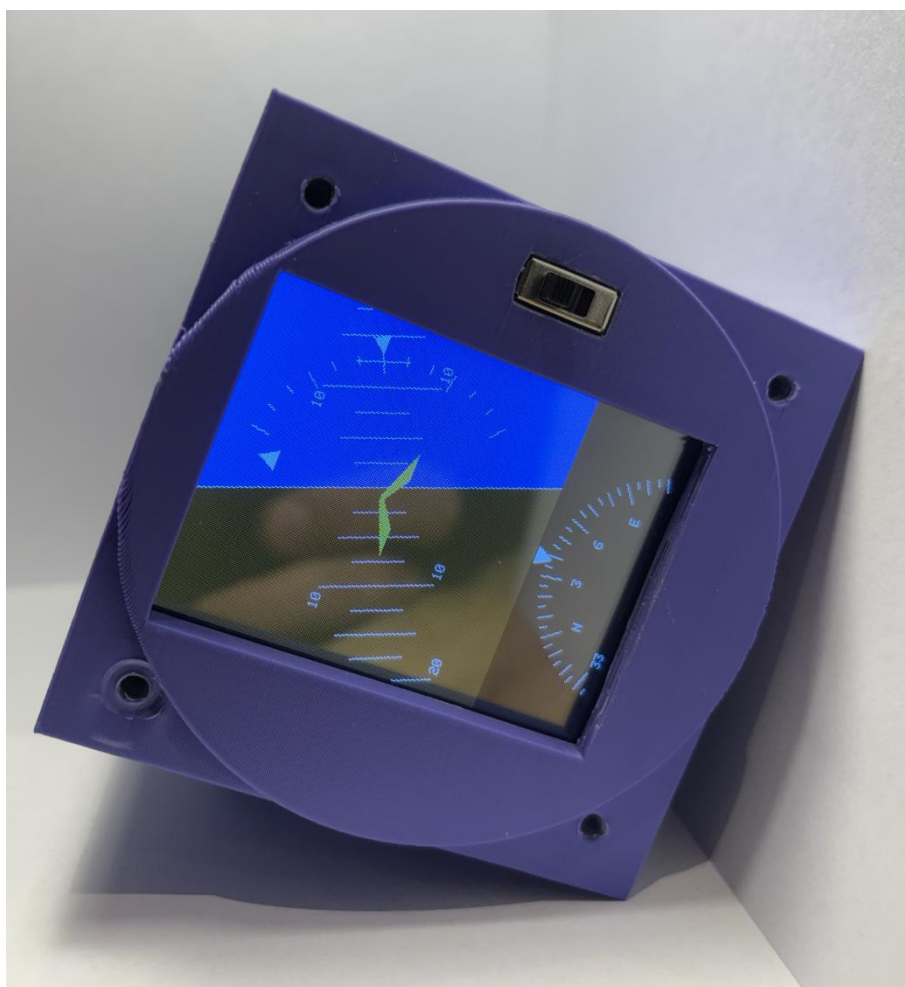
$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}. \quad (6)$$

9. Interfejs użytkownika na wyświetlaczu

Warstwa Display rysuje cały interfejs w pamięci za pomocą buforów GFXcanvas16, a następnie przesyła gotową bitmapę na ekran. Górna część wyświetlacza 240 x 240 px zajmuje sztuczny horyzont, a dolna część 240 x 80 px tworzy kompas.



Rys. 15. Widok sztucznego horyzontu w poziomie



Rys. 16. Widok sztucznego horyzontu przy przechyleniu

10.Podsumowanie i dalsze kierunki rozwoju

Projekt AHRS łączy warstwę sprzętową, obudowę, warstwę elektroniczną oraz oprogramowanie realizujące fuzję danych i wizualizację orientacji. Aktualna wersja korzysta ze sprawdzonej biblioteki Adafruit AHRS, co pozwala szybko uzyskać stabilne wskazania na prototypie. Równolegle w repozytorium istnieje autorski filtr Kalmana, który może stać się docelowym algorytmem po dopracowaniu jednostek, inicjalizacji i testów.

Załączniki

1. Kod źródłowy i dokumentacja projektu:

hiperłącze: github.com/kurowskitomek/AHRS

2. Nagrania z testów systemu w locie:

hiperłącze:

Wykaz rysunków

Rys. 1. Uproszczona architektura przepływu danych w projekcie	2
Rys. 2. Płytki Arduino Nano ESP32 wykorzystana jako jednostka sterująca.....	4
Rys. 3. Moduł IMU dostarczający pomiary inercyjne i magnetyczne.....	5
Rys. 4. Wyświetlacz LCD TFT używany do prezentacji wskazań AHRS	6
Rys. 5. Adapter dla Arduino Nano ESP32.....	6
Rys. 6. Przetwornica step-up Pololu U3V16F3	7
Rys. 7. Przełącznik suwakowy włączający sztuczny horyzont.....	7
Rys. 8. Rysunek wewnętrznej części obudowy	8
Rys. 9. Schemat połączeń płytki Arduino Nano ESP32	9
Rys. 10. Schemat zasilania projektu	10
Rys. 11. Schemat obszaru prototypowego	10
Rys. 12. Interfejs programu MotionCal	13
Rys. 13. Parametry kalibracji magnetometru.....	13
Rys. 14. Dane kalibracji żyroskopów	14
Rys. 15. Widok sztucznego horyzontu w poziomie.....	20
Rys. 16. Widok sztucznego horyzontu przy przechyleniu.....	21

Wykaz tabel

Tab. 1. Tabela komponentów projektu	4
Tab. 2. Tabela elementów oprogramowania	11

Wykaz fragmentów kodu

Fragment kodu 1. Tworzenie zadań na oddzielnych rdzeniach	12
Fragment kodu 2. Dane kalibracji w szkicu służącym do jej zapisu w pamięci EEPROM.....	14
Fragment kodu 3. Uzgodnienie osi IMU	15
Fragment kodu 4. Ustawienia IMU.....	16
Fragment kodu 5. Aktualizacja filtra Kalmana i pobranie danych.....	16

Wykaz równań

(1).....	18
(2).....	18
(3).....	18
(4).....	18
(5).....	19
(6).....	19